①

Sliding Window — String

Leet-438 Find all Anagrams in String

* Sliding window + Freq map / Hash map

string S = cbaebabacd string P = abc
↓ 0 ↓ 6
2 anagram
output → [0, 6]

Code →
```
if (P.size() > S.size)
    return {};
```
→ Size of P should be smaller than string S

```
vector<int> freqP (26,0)  freqS (26,0)
```
→ Hash array

```
for (char c : P)
    freqP [c - 'a']++;
```

'a' - 'a'
97 - 97 = 0 ↓ index

'b' - 'a'
98 - 97 = 1 ↓ index

refer leetcode

Algo

* Store P into Hash array
* use sliding window of size equal to P string

```
if (i-j+1 > P.size)
    freqS[S[j] - 'a']--;
    j++;
```

$$T - O(\underset{\text{length of P}}{m}+\underset{\text{length of S}}{n}) \simeq O(n)$$

$$S - O(26+26) = O(56) \simeq O(1)$$

Leet-3 Longest Substring Without Repeating Characters

optimal → sliding + Map → check leet

more optimal & tricky → sliding + Hash Array

```
vector<int> last (256, -1);
```
→ stores the index where a particular character was lastly seen

↳ Build Hash array of 256 characters with lastly stored index at -1

Ajanta PRODUCTS Officer

Window →

```
if (last[s[i]] >= j)
    j = last[s[i]] + 1;
last[s[i]] = i;
ans = max(ans, i-j+1);
```

if (last[s[i]] >= j) → using this we will get to know char already occured

j = last[s[i]] + 1; → move j ahead

last[s[i]] = i; → if char occured for 1st time store it's last seen index

Dry Run

String → abcabc

* i=0 s[i] = 'a' ~~freq~~ last['a'] = -1 → initially created hash array of 256 char with all at -1

j=0

last['a'] = 0;

Condition → if (freq['a'] >= j) ↓ no shrinking

Condition not matched

* i=1 s[i] = 'b' last['b'] = -1

last['b'] = 1

* i=2 s[i] = 'c' last['c'] = -1

last['c'] = 2

* i=3 s[i] = 'a' last['a'] = 0

Condition → if (last['a'] >= j) ↓ 0 ↓ 0

Shrink by moving j ahead → j = last['a']+1 ⇓ 0+1=1

j=1

* i=4 s[i] = 'b' last[b] = 1 j=1

j=2

* i=5 s[i] = 'c' last[c] = 2 → j=3

So on

Hash array

T — O(n)

S — O(1) {O(256)}

Map

T → O(n)

S — O(min(n, k)) ↳ size of char set.

㊖ Rising Brain — 3rd of string (GFG) → Longest Substring with K uniques

```
vector<int> freq(26, 0);
int dist = 0
int ans = -1
int j = 0
for (i = 0 to n)
    if (freq[s[i] - 'a']++ == 0)
        distinct++
    while (dist > K)
        if (-- freq[s[i] - 'a'] == 0)
            dist--;
        j++;
    if (dist == K)
        ans = max(i-j+1, ans);
```

larger version → if (freq[i] == 0) dist++ freq[i]++

→ means new distinct char arrived

→ mark it as seen already

T — O(n)

S — O(26) ~ O(1)

Shrink → while (dist > K)

Can also use unordered_map see code on GFG

T — O(n)

S — O(k)

Minimum Window Substring

Leet - 76

Tough

```
vector<int> freq(256, 0)          → only 1 Hasharray to track both strings

for(char c : t)                    } track the requirments (t string); which char
    freq[c]++                        need to be covered

int count = 0                      → track whether requirments (char of 't' string) got
                                     covered in our window or not
int sp = 0                         → 1st index of minimum string required
                                     (jth boundary | left boundary of window)

for (i=0 to s.size())

    if (freq(s[i] >0))             → current element is in hash means it's required
                                     char (char of 't' string) so need to be
                                     included in window
        count++                    → requirement fulfilled

    freq[s[i]]--;                  → requirement is fulfilled so decrement it
                                     (element of t got covered in window
                                      so remove from Hash)
    (unnecessary char which are not in t will get freq in -ve.)

    while (count == t.size())      → count tracks (increments) only when
                                     req. is fulfilled so when it get equal
                                     to size of 't' that means all
                                     char of 't' (all requirments) got covered
    (shrink window when all char of 't' got covered)

        if (i-j+1 < mind)          → update min window size
            mind = i-j+1
            sp = j

        freq[S[j]]++               → Remove S[j] (shrink window from left)
                                     (refer leet to understand this)
        (unnecessary chars will have 0 or -ve freq
         only req char will get freq >0)

        if (freq[S[j]] >0)         → while shrinking if required char (char of t string)
                                     get removed from window
            count--                → decrement count to show that
                                     number of fulfilled requirment got reduced

        j++                        → increment j after shrinking window (left boundary
                                     shifted)

if (mind == INT_MAX)               } if requirment not fulfilled
    return ""                        edge case

return s.substring(sp, mind);      → Find Ans
```

Time → O(n+m)

Space → O(1)

(#) Leet – 567 → Permutation in String {Refer Leet}

(#) Leet – 30 → Substring with Concatenation of all words

V. tough

Sliding window + HASH

Refer Leet Code

& ** Techdose white vid #30

Time – Space

n = s.length()

m = words.size()

L = length of each word

$T \Rightarrow O(n \times L + m \times L) \simeq O(n \times m$

$S \Rightarrow O(m \times L) \simeq O(m)$